

```
[ranan@Linux32 article]$ ./hackMe PASS          # [R1]
Authenticated ...program continuing...
[ranan@Linux32 article]$ ./hackMe FAIL          # [R2]
Wrong Input, exiting...
[ranan@Linux32 article]$ ./hackMe 123          # [R3]
Wrong Input, exiting...
[ranan@Linux32 article]$
```

The program continues execution only if the input string supplied as a command-line argument matches some criteria (in this case, a fixed string, 'PASS'). But what exactly causes it to fail in scenario R2 and scenario R3? Let's make use of *nm*—this command lists all the symbols in a binary.

```
[ranan@Linux32 article]$ nm hackMe | grep " T "
00048390 T authenticate
000484d4 T _fini
00048278 T _init
0004847c T __libc_csu_fini
0004844c T __libc_csu_init
000483c2 T main
000482e0 T _start
[ranan@Linux32 article]$
```

There are two functions of interest: *main* and *authenticate*. All the rest are provided by GCC. The developer didn't take enough care while naming—*authenticate* seems to be the weak function! It's a guess, but let's dig deeper.

Running the victim application in gdb: Mounting the attack manually

Let's apply the mechanism discussed in the basic concepts, and see if this application can be compromised! The session below shows the steps performed with *gdb* (I used v6.8):

```
[ranan@Linux32 article]$ gdb ./hackMe          (#1)
GNU gdb Red Hat Linux (5.3post-0.20021129.i386)
Copyright 2003 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License,
and you are welcome to change it and/or distribute copies of it
under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB. Type "show warranty"
for details.
This GDB was configured as "i386-redhat-linux-gnu"...
(gdb) br authenticate                          (#2)
Breakpoint 1 at 0x0048396
(gdb) r FAIL ----- > Program should
fail(#3)
Starting program: /home/rdeep/article/hackMe FAIL

Breakpoint 1, 0x0048396 in authenticate ()
(#4)
```

```
(gdb) stepi
0x0048396 in authenticate ()
(gdb) step
Single stepping until exit from function authenticate,
which has no line number information.
0x004840e in main ()
(gdb) p $eax
(#5)
$1 = 1
(gdb) set $eax=0 ----- > But we hacked!
(#6)
(gdb) p $eax
$2 = 0
(gdb) c
(#7)
Continuing.
```

```
Authenticated ...program continuing... ---- > And, reflected
here!
```

Before explaining what is happening in this session, let's recall some *gdb* commands:

- *stepi*—Step one assembly instruction at a time.
- *step*—Step program until it reaches a different source line; when the program is compiled without any debug information, this command stops just before coming out of the current function.
- *set*—Set the value of a variable or register, etc. *\$eax*, in this command, refers to the processor register *EAX*.
- To pass small (like int) return types, GCC uses the *EAX* processor register on the Intel platform.

Armed with this information, let's find out what is happening in the above session. I loaded the (release mode) program in *gdb* (#1), and inserted a breakpoint in the *authenticate* function (#2)—the one we need to manipulate. Then the program is started, and the command *r* (#3) provides a command-line argument, 'FAIL'. Since this is a wrong password, the program should end with a failure message. However, we intervene (with the help of the great *gdb*!) and alter the program flow. As soon as the program is run, it hits the breakpoint in the *authenticate* function (#4). We want control back with us as soon as *gdb* has finished executing *authenticate* and the program flow is just back into *main*. This is achieved by two *gdb* commands: *stepi* and *step*. By the time *step* completes, we are back in *main* and the return value of *authenticate* is available in the *EAX* processor register. As you can see, the return value was 1 (#5); we changed this to 0 (#6), and then we continued the program with the command *c* (#7)—and all this happened before copying the return value into the variable *retVal*. So, here is the result; we changed the normal execution flow of the program. The program that was supposed to fail has passed!